LAB ASSIGNMENT 2
Operating Systems (UCS-303)

Name: Divyank Rajput
Roll No.: 2026010038
Section: MCA-2

Q1. Introduction of Internal and External Commands.

Answer:

Linux commands can be broadly classified into internal commands and external commands.

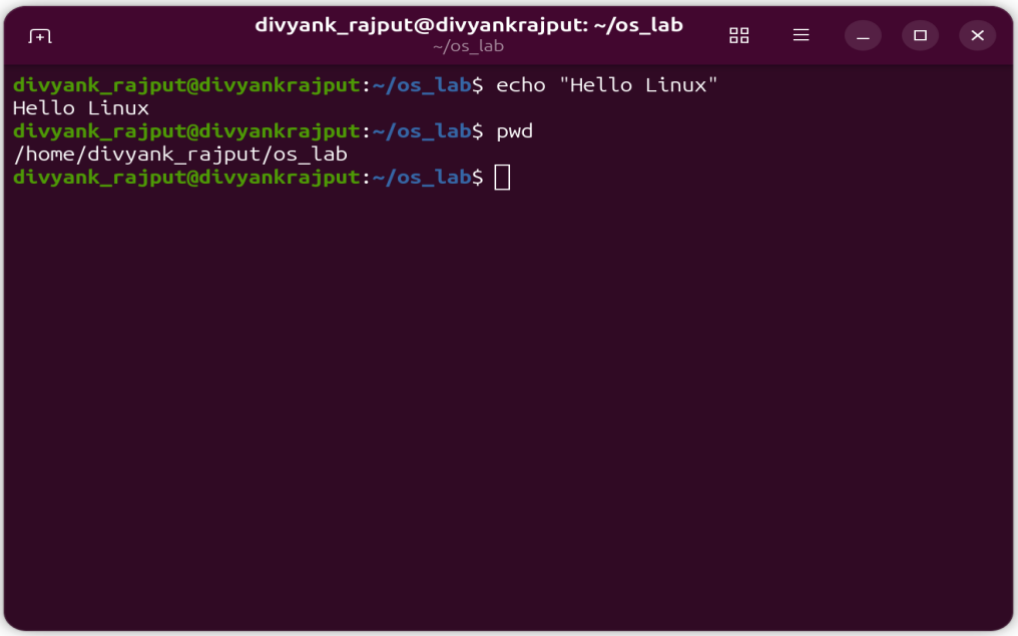
Internal Commands:
Internal commands are commands that are built into the shell itself. They do not require a separate executable file to run and are executed directly by the shell.

Examples:

· cd - Changes the current working directory.
· echo - Displays text or the value of a variable.
· pwd - Displays the current working directory.
· read - Accepts input from the user.
· export - Sets or exports environment variables.

Example:

```
echo "Hello Linux"
pwd
```



External Commands:
External commands are programs stored as separate executable files in the Linux file system. When an external command is executed, the shell searches for its executable file in directories specified by the PATH variable.

Examples:

· ls - Lists files and directories.

· cat - Displays file contents.

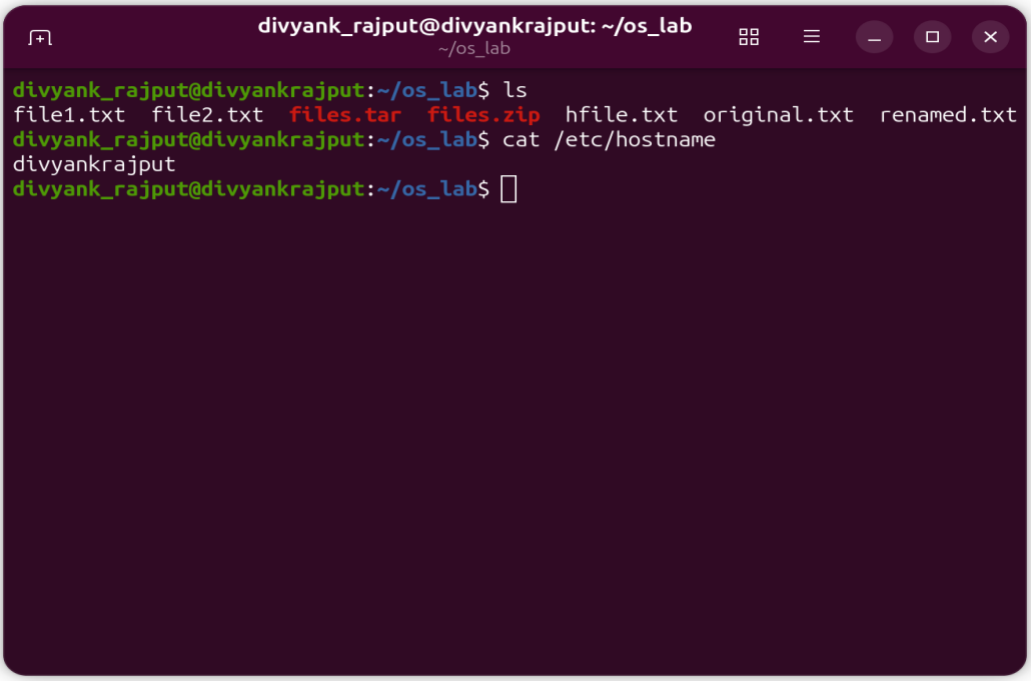
· cp - Copies files or directories.

· mv - Moves or renames files and directories.

· grep - Searches for a pattern in text.

Example:

ls
cat /etc/hostname



Q2. Difference between Internal and External Commands:

· Internal commands are built into the shell, whereas external commands are separate executable programs.

· Internal commands are executed directly by the shell, whereas external commands are located through the PATH.

· Examples of internal commands are cd and echo; examples of external commands are ls, cat and grep.

2. Feeding output of one command to another command by pipelining.

Answer:

A pipe (|) is used in Linux to send the output of one command as the input to another command. This process is called pipelining.

The general syntax is:

command1 | command2

For example:

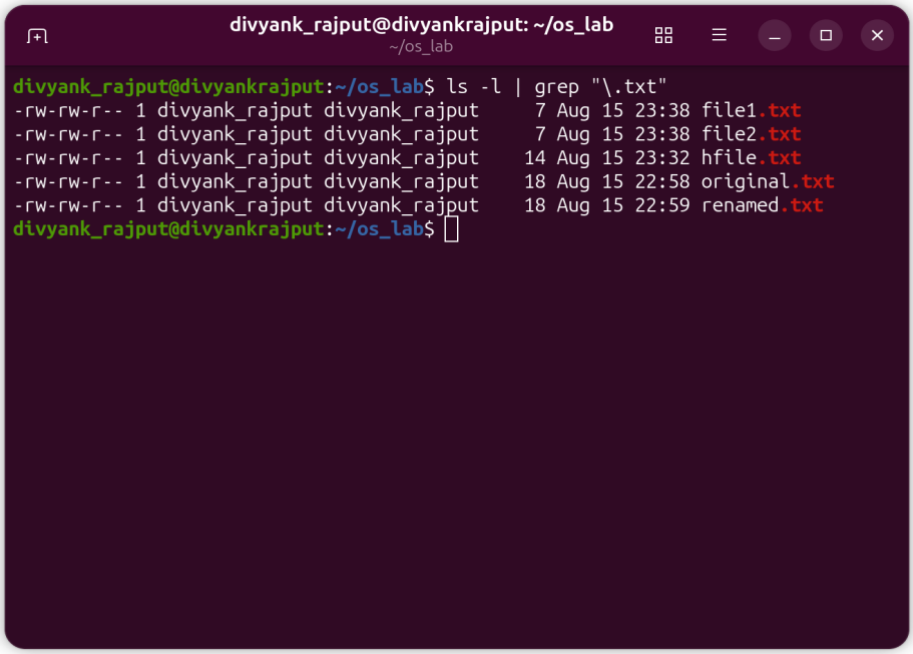
ls | wc -l

Here, ls produces a list of files and directories, and the pipe sends this output to wc -l, which counts the number of lines.

Another example is:

ls -l | grep ".txt"

In this command, ls -l displays the directory contents and grep filters the output to show entries containing .txt.



3. expr, locating command.

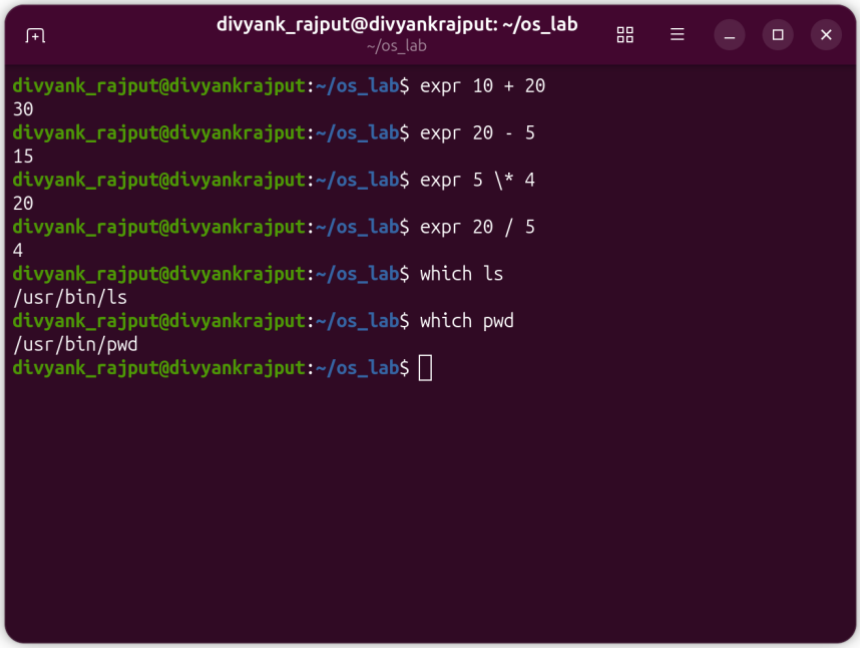
Answer:

expr Command:
The expr command is used to evaluate expressions, especially arithmetic expressions in shell scripts. Shell variables are treated as string variables, so expr can be used to perform arithmetic operations.

The terms of an expr expression must be separated by spaces. The multiplication operator * should be preceded by a backslash (\) so that the shell does not treat it as a wildcard character.

Examples:

expr 10 + 20
expr 20 - 5
expr 5 * 4
expr 20 / 4



Locating a Command:
The which command can be used to locate the executable associated with a command. It displays the path of the command executable.

which ls
which pwd

4. echo command.

Answer:

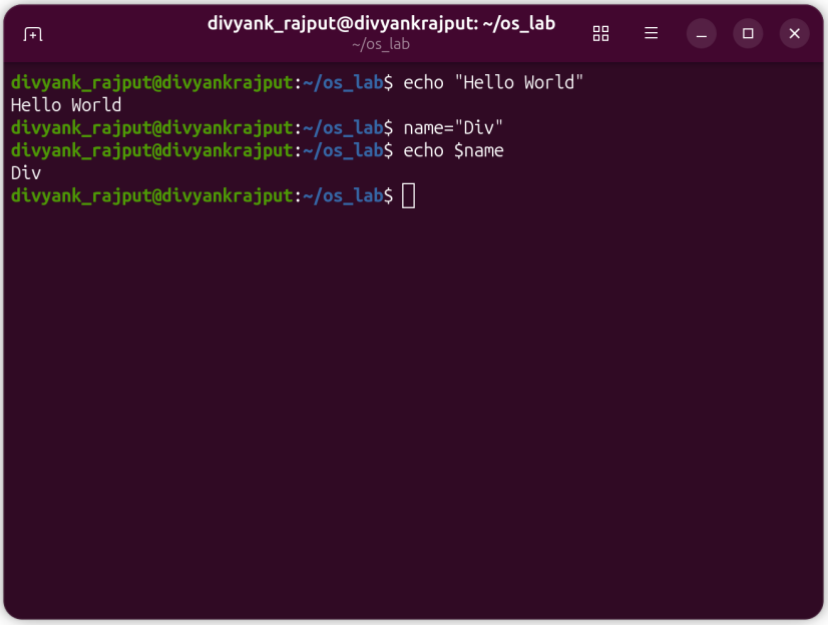
The echo command is used to display text or the value of a variable on the terminal. It is commonly used in shell scripts for displaying messages and variable values.

For example:

```
echo "Hello World"
```

The echo command can also be used to display the value of a variable:

```
name="Student"
echo $name
```



5. Using . and ..

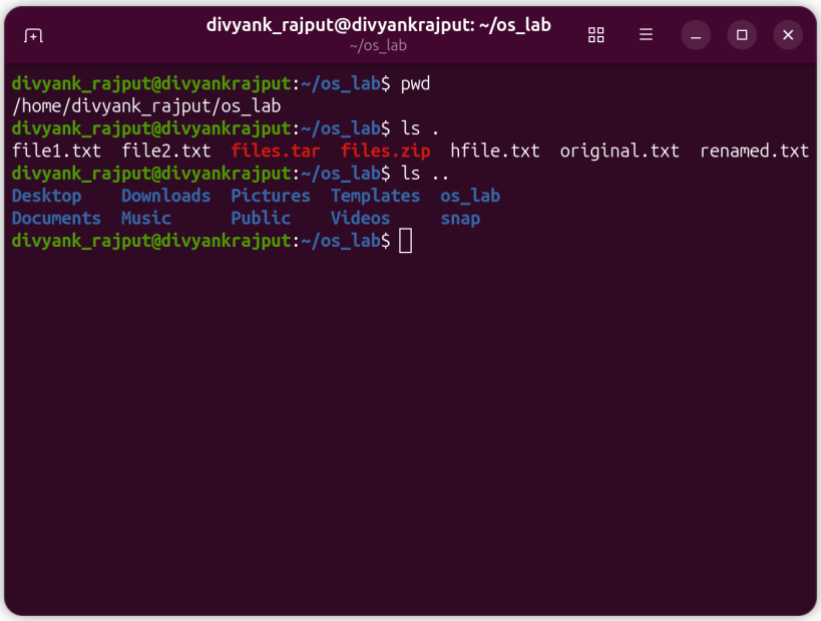
Answer:

In Linux, . and .. have special meanings when referring to directories.

. (single dot): Represents the current directory. For example, ls . lists the contents of the current directory.

.. (double dot): Represents the parent directory of the current directory. For example, ls .. lists the contents of the parent directory. The command cd .. can be used to move to the parent directory.

```
pwd
ls .
ls ..
```



6. Ways for signing off from Linux.

Answer:

A user can sign off or exit from a Linux terminal using the following methods:

1. exit - Terminates the current shell session.

2. logout - Logs the user out of a login shell.

3. Ctrl + D - Sends an end-of-file signal and can be used to exit the shell.

These methods are commonly used to end a Linux shell session.

Commands:

exit
logout

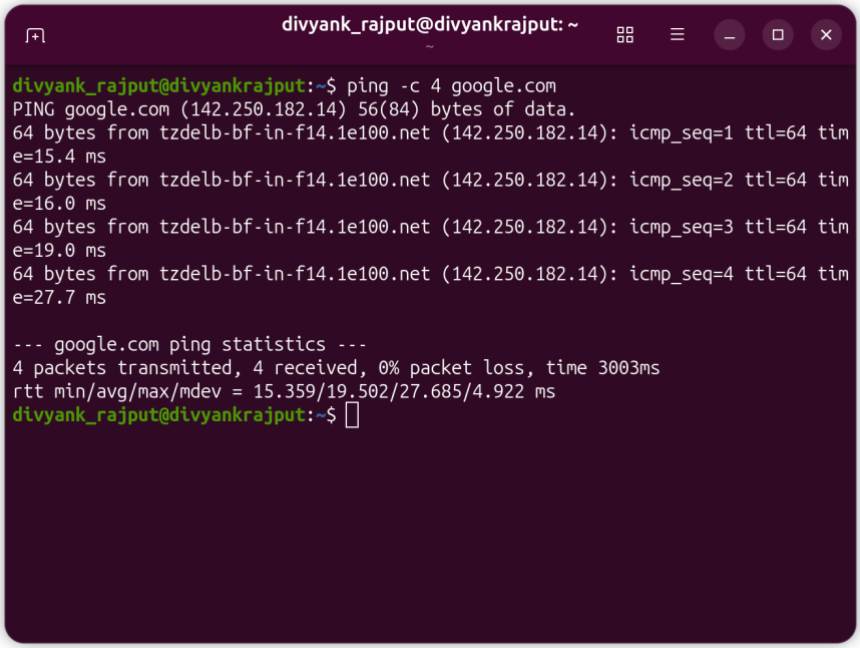
7. Ping, Man and help command.

Answer:

ping Command:

The ping command is used to test network connectivity between the computer and another host. It sends packets to the specified host and reports the response.

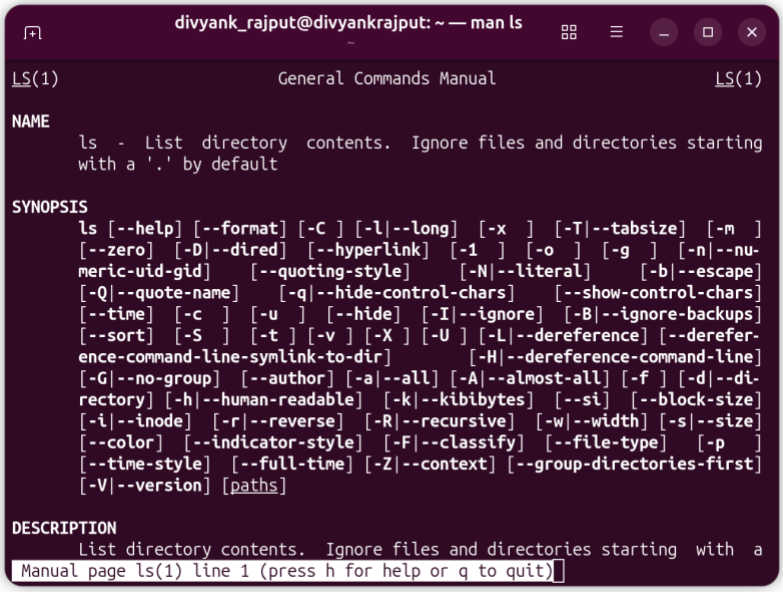
ping -c 4 google.com



man Command:

The man command displays the manual page of a Linux command. It provides information about the command, its options and usage. Press q to exit the manual page.

man ls

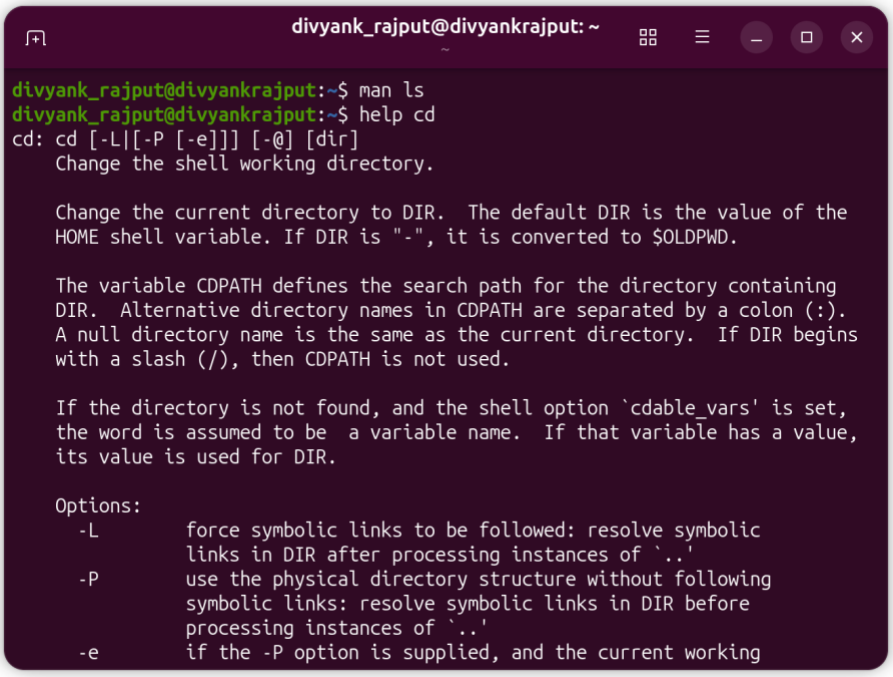


Answer:

help Command:

The help command provides information about shell built-in commands. For example:

help cd



8. Combining the commands.

Answer:

Multiple Linux commands can be combined to perform operations efficiently. Common operators used for combining commands are ;, && and ||.

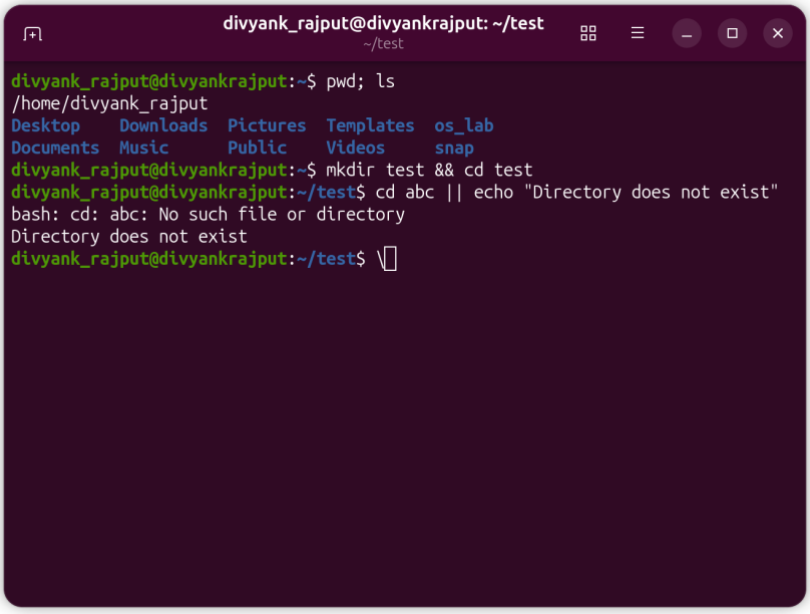
1. Semicolon (;) - Executes commands one after another, regardless of whether the previous command succeeds or fails.

2. AND operator (&&) - Executes the second command only if the first command is successful.

3. OR operator (||) - Executes the second command if the first command fails.

Examples:

pwd; ls
mkdir test && cd test
cd abc || echo "Directory does not exist"



9. File permissions and changing the access rights (chmod).

Answer:

Linux provides file permissions to control who can read, write or execute a file. There are three basic file permissions:

· r (Read) - Permission to read the file.

· w (Write) - Permission to modify the file.

· x (Execute) - Permission to execute the file.

Permissions are assigned to three categories:

· User (u) - The owner of the file.

· Group (g) - The group associated with the file.

· Others (o) - All other users.

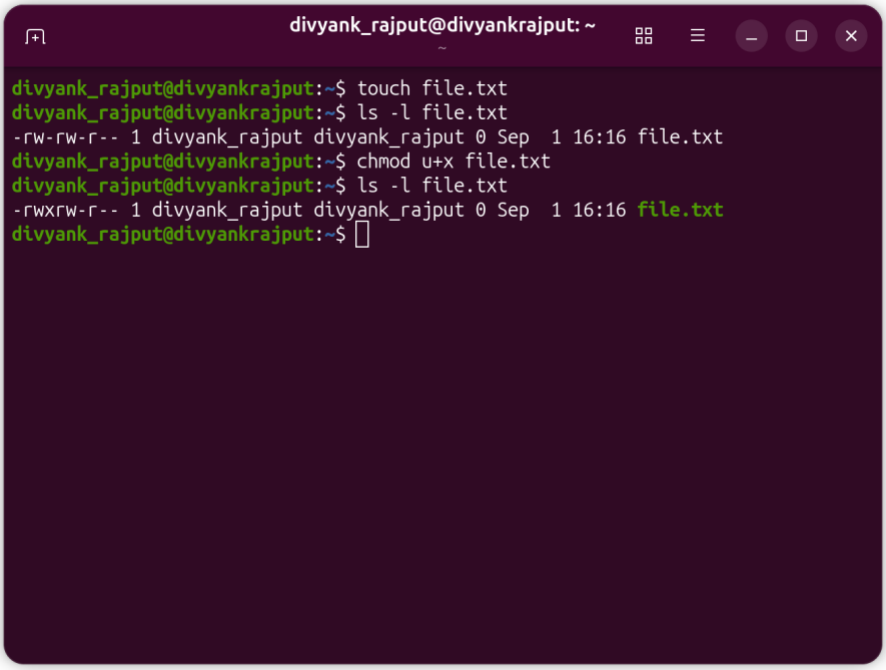
The ls -l command is used to view file permissions. The chmod command is used to change file access permissions.

```
touch file.txt
ls -l file.txt
chmod u+x file.txt
ls -l file.txt
```

Numeric Permissions:
Linux permissions can also be represented using numbers. Read has a value of 4, write has a value of 2 and execute has a value of 1.

For example, chmod 755 file.txt gives the owner read, write and execute permissions, while the group and others get read and execute permissions.

```
chmod 755 file.txt
ls -l file.txt
```



10. vi editor and its basics: write a small paragraph using vi editor.

Answer:

The vi editor is a text editor available in Linux. It can be used to create and edit text files.

The two basic modes are:

1. Command Mode - Used for executing editor commands.

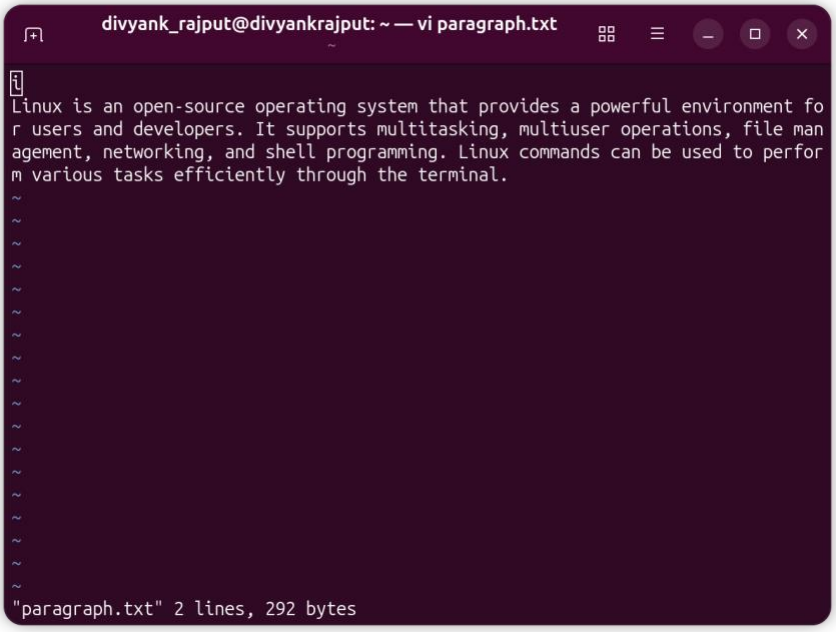
2. Insert Mode - Used for entering text into the file.

Steps to write a paragraph:
1. Open a file using vi.
2. Press i to enter Insert Mode.
3. Type the paragraph.
4. Press Esc to return to Command Mode.
5. Type :wq and press Enter to save the file and quit.

vi paragraph.txt

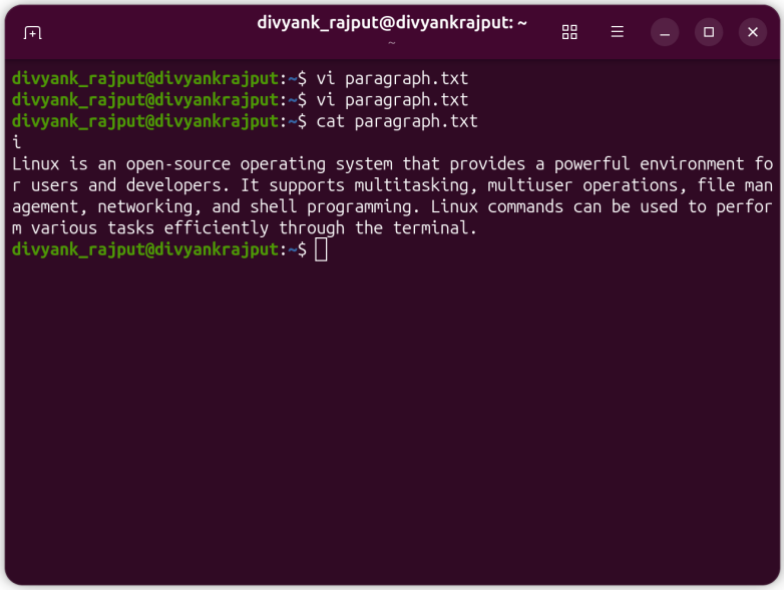
Paragraph to type in vi:

Linux is an open-source operating system that provides a powerful environment for users and developers. It supports multitasking, multiuser operations, file management, networking, and shell programming. Linux commands can be used to perform various tasks efficiently through the terminal.



After saving the file, verify its contents using:

cat paragraph.txt



SELF STUDY

1. grep, sort, ps command

Answer:

grep Command:
grep is used to search for a specified pattern in text.

sort Command:
sort is used to sort lines of text.

ps Command:
ps displays information about currently running processes.

Example for grep:

```
echo "Linux Operating System" > data.txt
echo "Shell Programming" >> data.txt
echo "Linux Commands" >> data.txt
grep "Linux" data.txt
```

Example for sort:

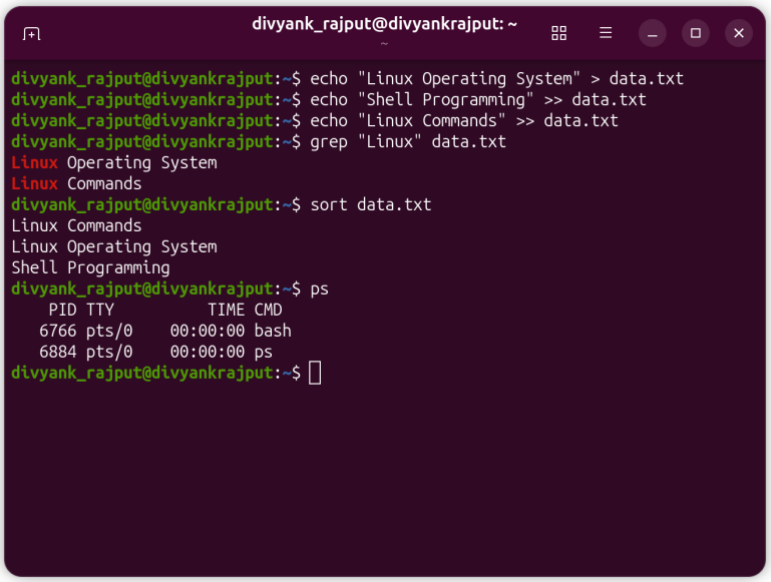
```
sort data.txt
```

SCREENSHOT – sort command output

[INSERT SCREENSHOT HERE]

Example for ps:

```
ps
ps -ef
```



2. Study Basics of shell programming.

Answer:

A shell program is a series of Unix/Linux commands written together to perform a particular task. Instead of specifying one job at a time, the shell is given a to-do list of commands that carries out an entire procedure. Such programs are known as shell scripts.

Shell programming provides features used in programming, such as variables, input/output, arithmetic operations, decision statements and loops.

Shell Variables:

· A variable name can contain alphabets, digits and an underscore (_).

· No commas or blanks are allowed within a variable name.

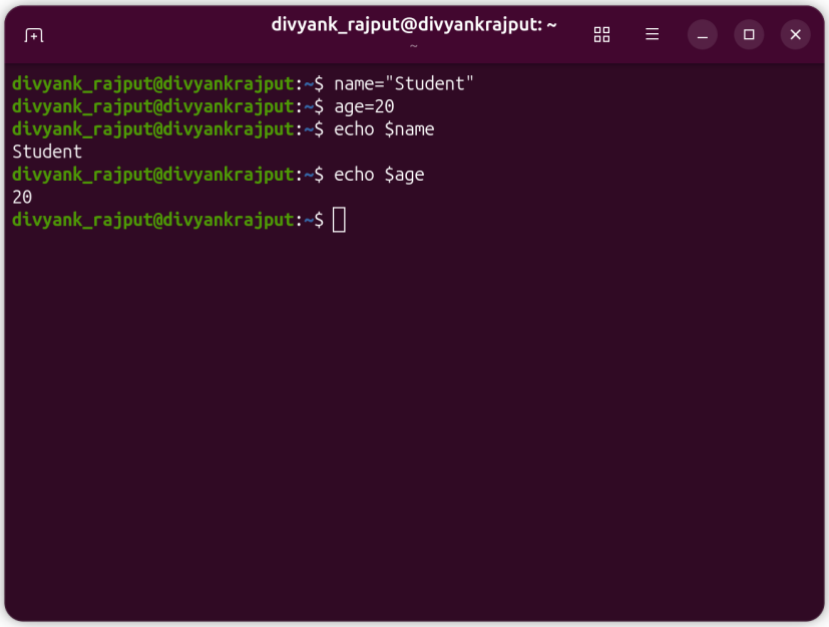
· The first character must be an alphabet or an underscore.

· Variable names should be of any reasonable length.

· Variable names are case-sensitive.

Example:

```
name="Student"
age=20
echo $name
echo $age
```



Input/Output:

Answer:

The read command is used for accepting input from the user, while echo is used for displaying output. Values can be assigned to variables using the read statement or a simple assignment operator. No spaces should be given on either side of = during assignment. The $ symbol is used to access the value of a variable.

Arithmetic in Shell Script:

Shell variables are string variables, so the expr command can be used to evaluate arithmetic expressions. The terms of the expression must be separated by spaces. For multiplication, the * symbol should be preceded by a backslash.

```
#!/bin/bash

a=10
b=5

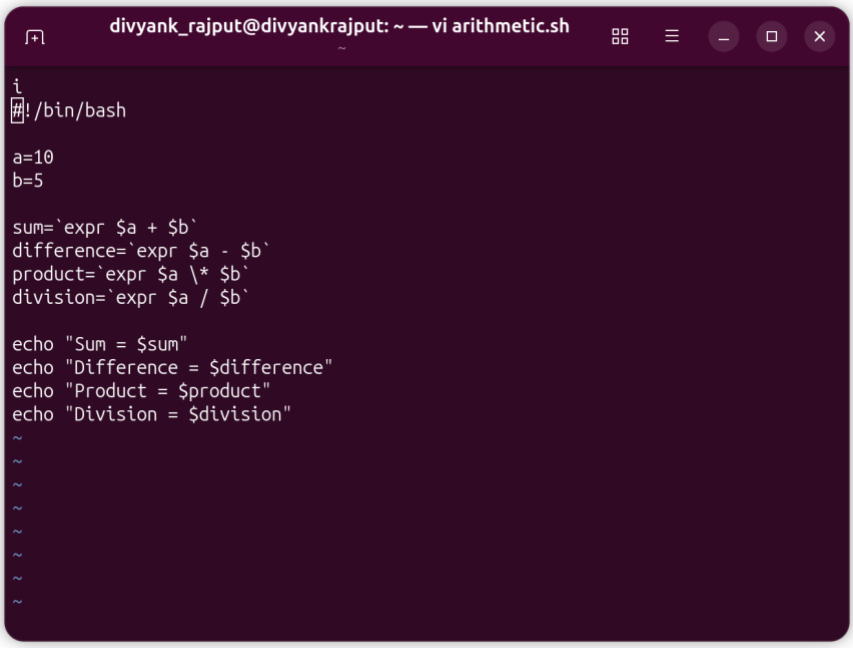
sum=`expr $a + $b`
difference=`expr $a - $b`
product=`expr $a \* $b`
division=`expr $a / $b`

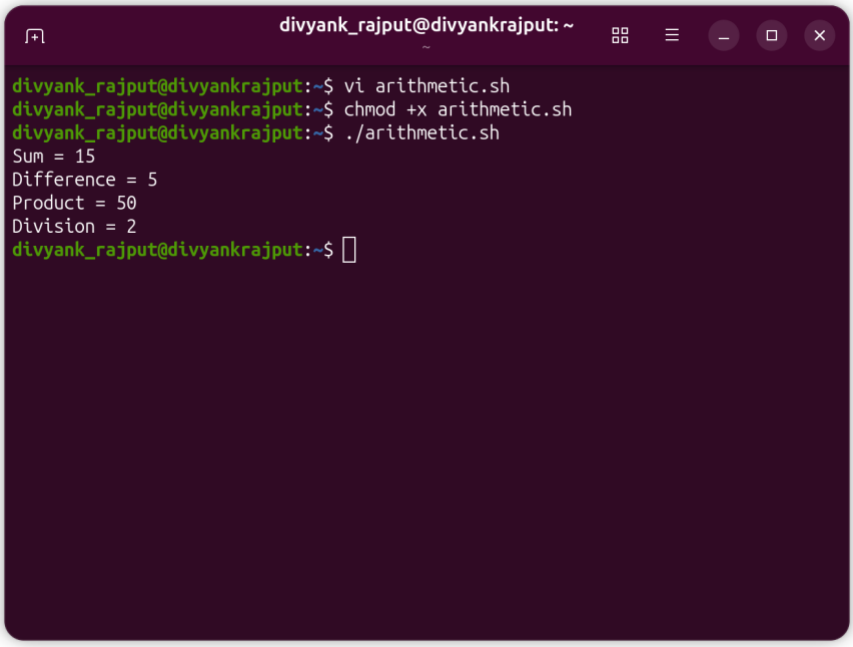
echo "Sum = $sum"
echo "Difference = $difference"
echo "Product = $product"
echo "Division = $division"
```

To execute the script:

chmod +x arithmetic.sh

./arithmetic.sh





Operators Used in Shell Script:

· -gt - Greater than

· -lt - Less than

· -ge - Greater than or equal to

· -le - Less than or equal to

· -ne - Not equal to

· -eq - Equal to

· -a - Logical AND

· -o - Logical OR

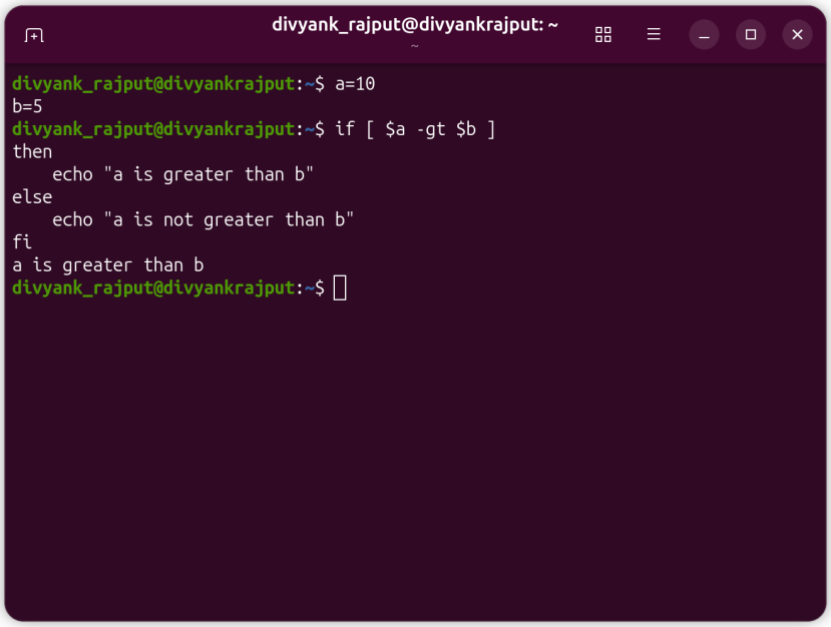
· ! - Logical NOT

Example using a comparison operator:

```
#!/bin/bash

a=10
b=5

if [ $a -gt $b ]
then
   echo "a is greater than b"
else
   echo "a is not greater than b"
fi
```



Control Instructions in Shell:

Answer:

There are four types of control instructions in shell:

1. Sequence Control Instruction.
2. Selection or Decision Control Instruction.
3. Repetition or Loop Control Instruction.
4. Case Control Instruction.

Decision statements – If-then-else-fi

Answer:

The if-then-else-fi statement is used to make a decision based on a condition.

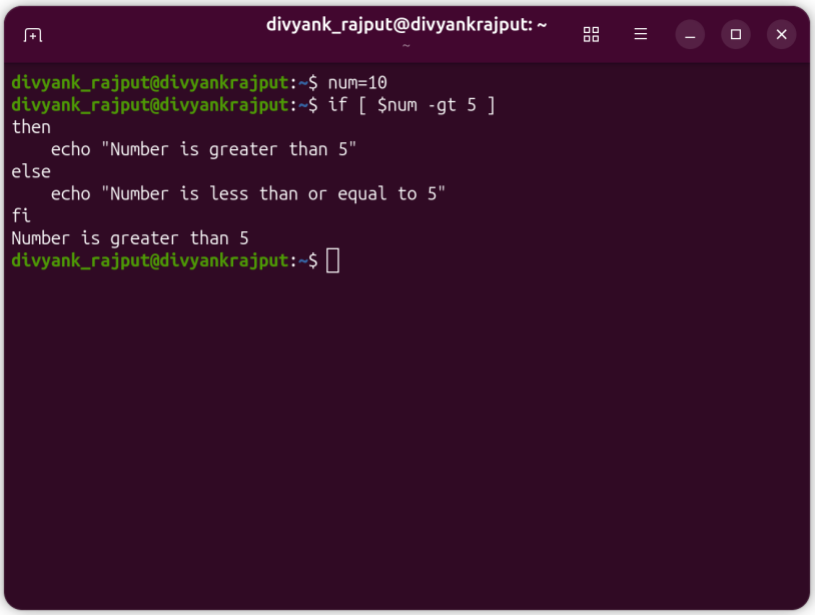
```
if condition
then
   Commands
else
   Commands
fi
```

Example:

```
#!/bin/bash

num=10

if [ $num -gt 5 ]
then
   echo "Number is greater than 5"
else
   echo "Number is less than or equal to 5"
fi
```



For statement

Answer:

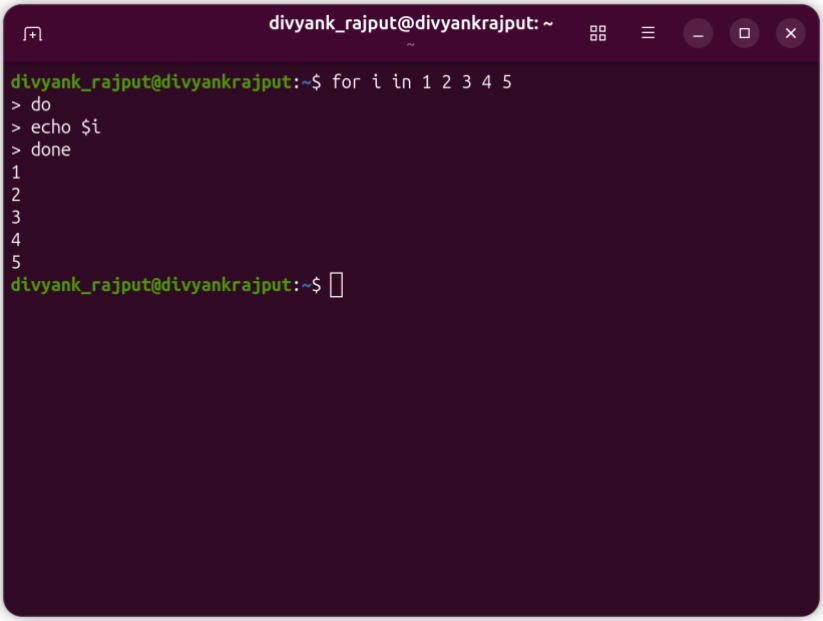
The for statement is used to repeat a command list for each value in a specified list.

```
for control_variable in value1 value2 value3
do
   Command list
done
```

Example:

```
#!/bin/bash

for i in 1 2 3 4 5
do
   echo $i
done
```



While statement

Answer:

The while statement repeatedly executes a command list as long as the specified condition remains true.

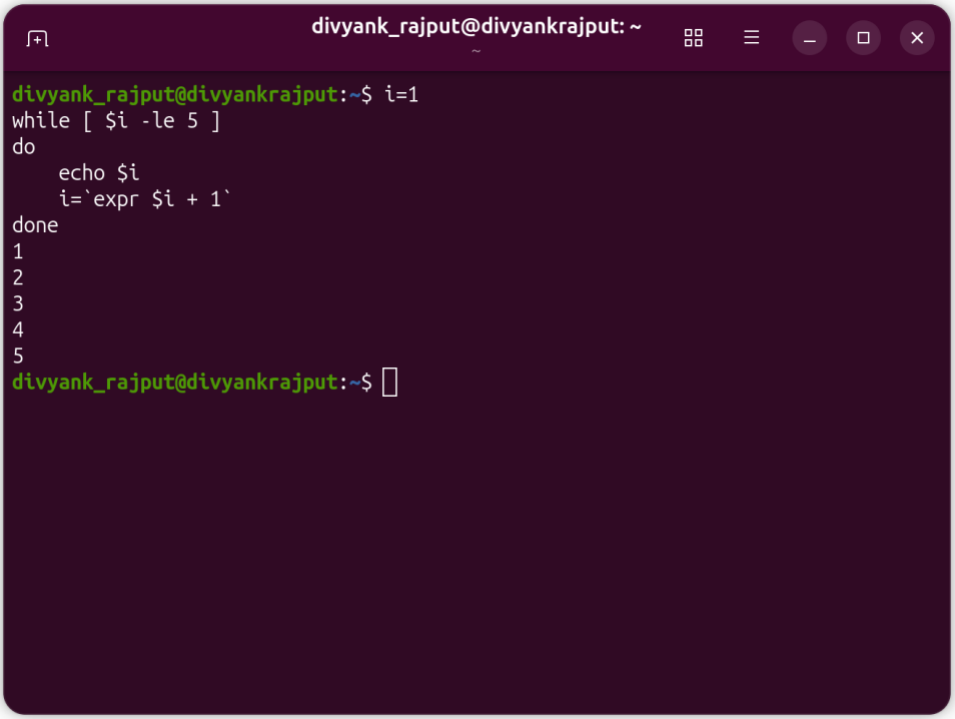
```
while control command
do
   Command list
done
```

Example:

```
#!/bin/bash

i=1

while [ $i -le 5 ]
do
   echo $i
   i=`expr $i + 1`
done
```



Until statement

Answer:

The until statement repeatedly executes a command list until the specified condition becomes true.

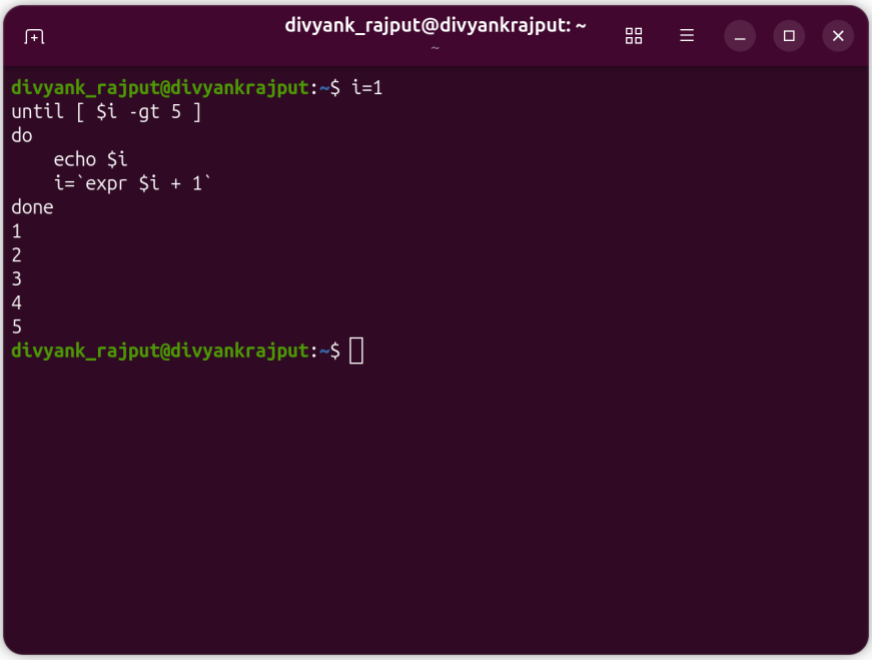
```
until control command
do
   Command list
done
```

Example:

```
#!/bin/bash

i=1

until [ $i -gt 5 ]
do
   echo $i
   i=`expr $i + 1`
done
```



Case statement

Answer:

The case statement is used when a value has to be compared with multiple choices. The matching choice is executed.

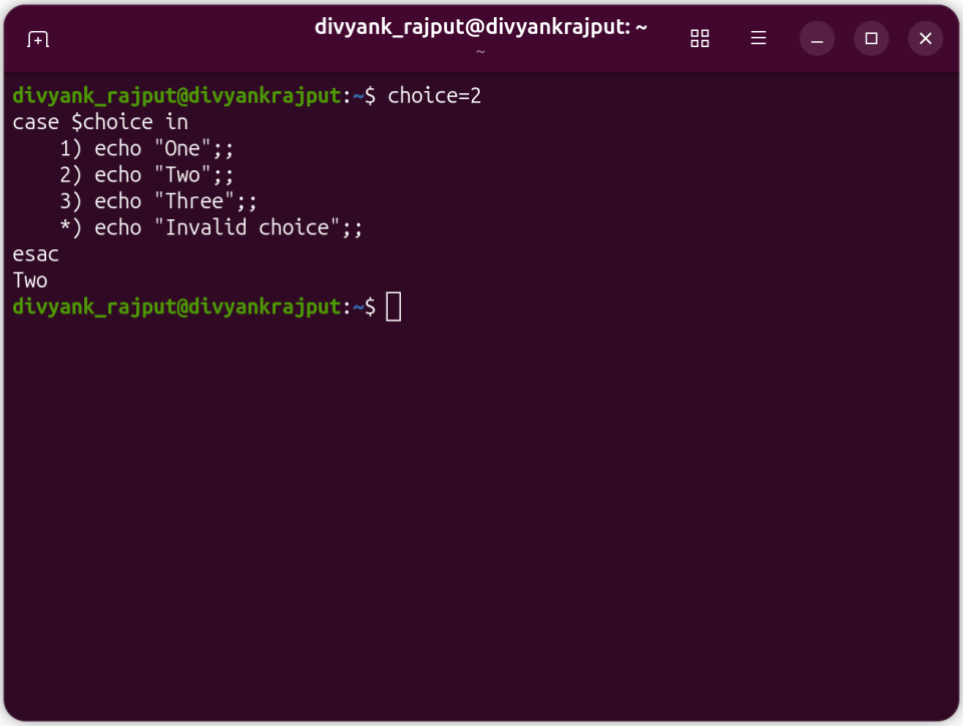
```
case value in
  choice 1) commands;;
  choice 2) commands;;
esac
```

Example:

```
#!/bin/bash

choice=2

case $choice in
  1) echo "One";;
  2) echo "Two";;
  3) echo "Three";;
  *) echo "Invalid choice";;
esac
```



Steps to write and execute a script

Answer:

The steps to write and execute a shell script are:

1. Open the terminal and go to the directory where you want to create the script.
2. Create a file with .sh extension.
3. Write the script in the file using an editor.
4. Make the script executable using chmod +x <fileName>.
5. Run the script using ./<fileName>.

chmod +x <fileName>
./<fileName>